Group.26.ino

```cpp
#include <LiquidCrystal.h>
#include <Keypad.h>
#include <Servo.h>

// --- Hardware Initialization ---
// Pins for LCD: RS, En, D4, D5, D6, D7
LiquidCrystal lcd(8, 9, 10, 11, 12, 13);
Servo srv;

// Buzzer on A0 (14), Servo Signal on A1 (15)
const int bz = 14, srvP = 15;

// Keypad Configuration
const byte R = 4, C = 4;
char keys[R][C] = {
  {'1','2','3','A'},
  {'4','5','6','B'},
  {'7','8','9','C'},
  {'*','0','#','D'}
};
// Mapping Keypad pins to Arduino (A2, A3, D2, D3 for rows / D4-D7 for cols)
byte rP[R] = {16, 17, 2, 3}, cP[C] = {4, 5, 6, 7};
Keypad kpd = Keypad(makeKeymap(keys), rP, cP, R, C);

```

```cpp
// --- Global Variables ---
char pass[16], in[16];   // Arrays to store the master password and user input
int idx = 0, errs = 0;   // idx tracks typing progress, errs tracks failed attempts
bool set = false;        // Track if a password has been established yet

void setup() {
  lcd.begin(16, 2);      // Initialize 16x2 LCD
  pinMode(bz, OUTPUT);   // Set buzzer as output
  srv.attach(srvP);      // Connect servo to pin
  Serial.begin(9600);    // Start serial for Bluetooth/Debugging

  move(false);           // Ensure door starts in LOCKED position
  reset("SET PASSWORD:"); // Initial prompt for the user
}

void loop() {
  // --- Bluetooth/Serial Remote Control ---
  if (Serial.available()) {
    char c = Serial.read();
    if (c == '0') errs = 3;                            // Remote Lockdown
    if (c == '1') { errs = 0; reset("SYSTEM READY"); } // Lift Lockdown
    if (c == '2') unlock();                            // Remote Open
    if (c == '3') { move(false); reset("LOCKED"); } // Remote Close
  }

```

```
  // --- Security Lockdown Logic ---
  if (errs >= 3) {
    lcd.setCursor(0,0); lcd.print("!! LOCKDOWN !!");
    tone(bz, 1000, 200); // Pulse alarm sound
    delay(500);
    return; // Skip the rest of the loop while in lockdown
  }

  // --- Keypad Input Logic ---
  char k = kpd.getKey();
  if (k) {
    if (k == '*') {
      // Clear current typing and show prompt
      reset(set ? "ENTER PASS:" : "SET PASS:");
    }
    else if (k == '#') {
      in[idx] = '\0'; // Null-terminate the input string for comparison

      if (!set) {
        // Logic for first-time password setup
        strcpy(pass, in);
        set = true;
        reset("ENTER PASSWORD:");
      }
      else if (strcmp(in, pass) == 0) {
```

Group.26.ino

```
        // Correct Password entered
        unlock();
      }
      else {
        // Wrong Password entered
        errs++;
        tone(bz, 150, 800); // Low pitch error sound
        reset("WRONG! RETRY:");
      }
    }
    else if (idx < 15) {
      // Record keypress and mask it with '*' on the LCD
      in[idx++] = k;
      lcd.setCursor(idx-1, 1);
      lcd.print('*');
      tone(bz, 2000, 50); // Feedback "click" sound
    }
  }
}

// --- Door Actions ---

void unlock() {
  lcd.clear();
  lcd.print("OPEN");
```

```
  lcd.clear();
  lcd.print("OPEN");
  errs = 0;           // Reset error counter on success
  move(true);         // Rotate servo to open
  for(int i=0; i<3; i++) { tone(bz, 1200, 100); delay(200); } // Success beeps
  delay(2000);        // Keep door open for 2 seconds
  move(false);        // Auto-relock
  reset("ENTER PASS:");
}

// Controls Servo rotation (Open: 15 to 100 degrees | Close: 100 to 15 degrees)
void move(bool open) {
  int start = open ? 15 : 100, end = open ? 100 : 15;
  for (int p = start; open ? p <= end : p >= end; open ? p++ : p--) {
    srv.write(p);
    delay(10); // Smooth movement delay
  }
}

// Clears memory and prepares LCD for new input
void reset(const char* m) {
  idx = 0;
  memset(in, 0, sizeof(in)); // Wipe the input buffer
  lcd.clear();
  lcd.print(m);
```